

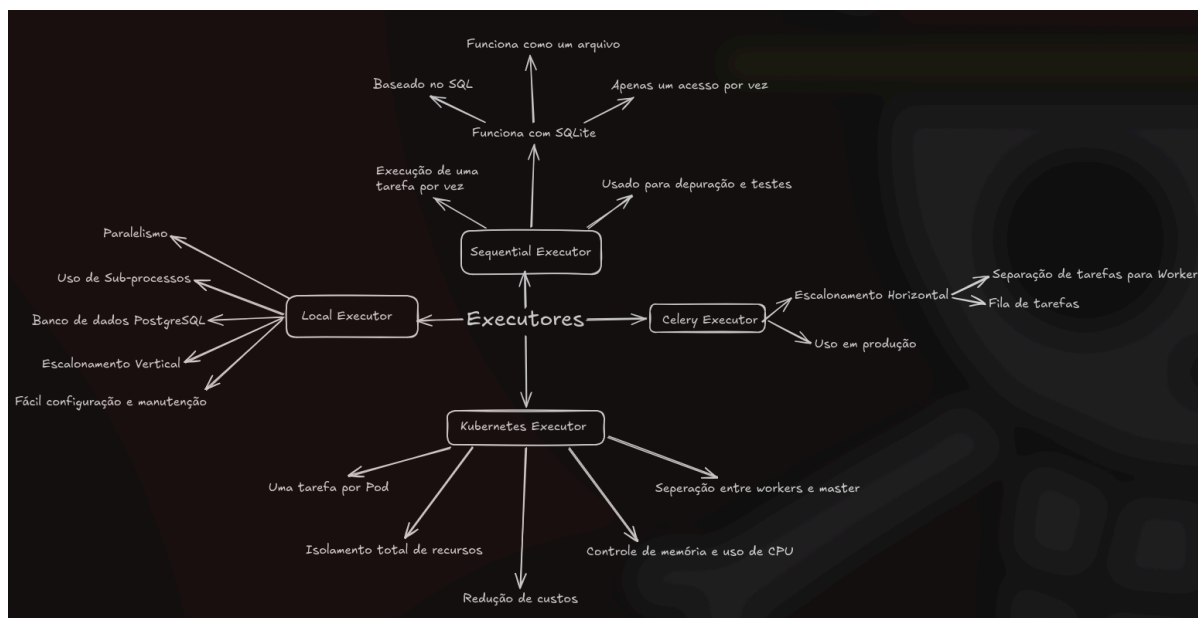
Relatório 19 - Pipelines de Dados II - Airflow (III)

Felipe Fonseca

1. Introdução

Nesse Card foram ensinados os conceitos de diferentes formas de executar tarefas no Airflow através de Executors, além de conceitos importantes que serão úteis como Branching, Templating, e XComs, além de apresentar técnicas de segurança do airflow como criptografia de dados sensíveis.

2. Desenvolvimento



Inicialmente foram apresentadas diferentes formas de executar tarefas no Airflow. Começando com o Sequential Executor, onde as tarefas rodam uma depois da outra em sequência, e o Local Executor, que permite que sejam criadas tarefas que rodam em paralelo uma à outra. Também foi apresentado o Celery Executor, que permite escalar os recursos e o Kubernetes Executor, que utiliza até mesmo virtualbox, sendo o mais complexo de entender.

Trabalhando com o Sequential Executor, o apresentador introduziu o SQLite, que é uma versão lite do SQL, que roda em um único arquivo e que é mais fácil de configurar, além de ter uma operação limitada.

Trabalhando depois com o Local Executor, foi apresentado um outro banco de dados mais robusto que consegue lidar com o paralelismo de atividades, que é o PostgreSQL. Dentro dessa configuração, foram apresentadas 3 variáveis principais para configuração da mesma, o `parallelism`, que define o limite máximo de Dags rodando simultaneamente no Airflow, o `max_active_run_per_dag`, que é a quantidade máxima de dags ativas ao mesmo tempo no projeto, e `dag_concurrency`, que é a quantidade de tarefas rodando dentro de uma mesma dag.

Após realizar testes com o Executor Local e o Sequential, é feita uma introdução ao Celery Executor. A ideia do Celery é lidar com o escalonamento, ou seja, o aumento no volume de dados. Ele faz isso distribuindo o trabalho para Workers, ou seja, trabalhadores, com ajuda de um notificador, o Message Broker. O funcionamento é: tarefas são enviadas para uma fila com sua descrição, essa fila é gerenciada pelo message broker, os Workers escutam essa fila e puxam as mensagens para executar as tarefas. A arquitetura do Celery é superior justamente por esse paralelismo, pois se um Worker quebra, outro pega a tarefa da fila, mas como consequência, o Celery também possui uma complexidade maior na hora de fazer manutenções.

Na prática, o instrutor ensinou a utilização do Flower para monitorar o Celery, olhando dados como Load Average para ficar de olho nos núcleos de CPU utilizados, e o `worker_concurrency`, para ver o número de tarefas que cada worker pode executar. Ele também mostrou o escalonamento horizontal com o Celery Executor, o recursos de Filas, e Pools. Este último é o mais interessante, que delimita a quantidade de acessos que tarefas podem ter em uma infraestrutura, basicamente criando concorrência entre as tarefas e podendo reorganizar a ordem conforme pesos (ou seja, prioridade das tarefas).

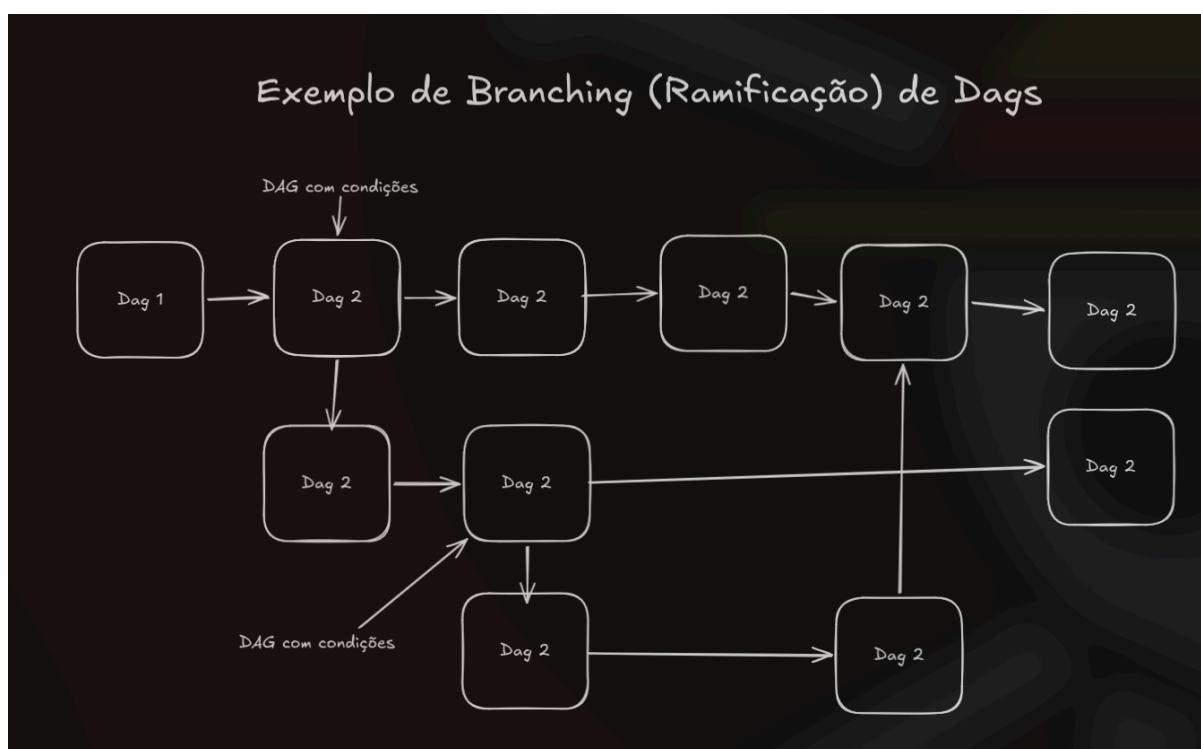
Por último na seção 5, foram apresentados os Kubernetes. Kubernetes é uma plataforma de código aberto criada pelo Google para organizar containers para automatizar as implementações. A arquitetura dos kubernetes pode ser resumida em Master Node e Workers Node. O master node é onde funciona o plano de controle, ele é quem faz todas as tarefas de gerenciar o sistema, tomar decisões, agendar containers e cuidar das requisições de API. Enquanto isso, os workers nodes são as que realizam as tarefas mesmo, seguindo as orientações do master node.

Os kubernetes resolvem um dos problemas do Celery, que é o de desperdício de recurso, tendo isolamento maior de falhas, controle exato de CPU e memória, e dessa forma garantindo que não tenha quase nada de consumo de processamento quando não tem dags rodando.

Iniciando na seção 6, o instrutor apresentou o conceito de SubDAGs, que é útil para quando se existem Dags executando funções muito semelhantes em paralelo uma a outra, nesse caso, podemos agrupá-las em uma subdag, todas gerenciadas por

apenas uma Dag. O problema das subdags são chamados de Deadlocks, que são travamentos causados por falta de espaço, por isso é necessário cuidado ao trabalhar com elas. As soluções apresentadas pelo apresentador são utilizar o Sequential Executor para as Subdags, o que faz com que elas rodem uma de cada vez e não esgotando os slots de memória, ou então criar uma fila de prioridade de execução, controlando assim a memória utilizada.

A parte mais interessante dessa seção foi a apresentação do Branching, ou ramificação. Através do BranchPythonOperator, podemos criar múltiplos caminhos a serem seguidos de acordo com variadas condições. O instrutor faz um exercício prático construindo dependências entre algumas tarefas para testar ramificações diferentes de resultados finais.



A próxima parte da seção foi para tratar do escalonamento dos dados no sentido de evitar quando possível valores estáticos no código como data, pois quando uma dag for executada ela buscaria sempre a mesma data engessada, e em alguns casos como pesquisa diária ou pesquisa SQL isso não é interessante.

Para lidar com isso foram criados 3 conceitos:

Variáveis: objetos ou valores armazenados no bd do airflow, é possível criptografar essa variável e é possível armazenar até objetos json inteiros dentro de uma variável, facilitando o gerenciamento de configurações complexas.

Templating: é um processo de injetar dados externos de forma dinâmica no código no momento em que a DAG está rodando.

Macros: são os atalhos que podemos injetar dentro de chaves do template para manipular dados.

Juntando esses 3 conceitos é possível fazermos códigos mais dinâmicos que podem ser alterados externamente, sem nada fixo na DAG, trazendo mais flexibilidade e escalabilidade para a arquitetura do Airflow.

O próximo conceito apresentado foi o XComs, que significa Cross-Communication. os Xcoms existem apenas por um motivo: compartilhar parâmetros pequenos entre as tarefas, que por padrão são isoladas. Foram feitos alguns testes utilizando push para enviar as mensagens e pull para receber os valores.

Depois foi apresentado o TriggerDagRunOperator, uma forma alternativa para não ter de utilizar subdags. De forma resumida, essa classe permite acionar uma Dag a partir de outra, onde uma envia as mensagens e parâmetros para a dag a ser acionada, e logo em seguida ela finaliza, deixando que a outra dag se vire sozinha. Ou seja, diferente das subdags, aqui é tudo independente uma Dag da outra.

E por fim dessa seção, foi apresentado o método contrário ao Trigger Dag, que é o External Task Sensor, que é ideal para criar dependência entre dags independentes. Isto é feito através de um sensor, que irá pausar a execução da Dag, ativar a outra Dag, esperar a finalização da outra Dag e só então continuar a execução.

Na seção 7 não teve nada de teoria, foi apenas uma demonstração da utilização do Airflow na Nuvem utilizando o Amazon EKS e a plataforma rancher. Lá ele mostrou como criar um servidor para o Rancher configurando porta, firewall, chave ssh, instalação do docker, configurações de segurança do Airflow, configuração de repositório, criação do Cluster Kubernetes e configurações, instalação do Nginx e do Airflow e um teste para verificar o funcionamento.

Na seção 8 o foco é totalmente no monitoramento do Airflow. O apresentador começa falando sobre o sistema de logs do Airflow, que é próprio de cada um dos componentes e que são armazenados em disco, o apresentador ensina a alterar as configurações de salvamento das logs através do arquivo de configuração do airflow.

Depois é apresentado o Elasticsearch, que é uma ferramenta open-source que pode fazer buscas de grandes volumes de dados salvos em Json, trabalhando em conjunto com o Stack ELK, que é uma ferramenta para construção de dashboards, ele possibilita visualizar muitos dados do funcionamento do Airflow, principalmente dados quantitativos, que não são obtidos diretamente pelas logs, como saúde do scheduler, uso de memória e capacidade, rastrear o comportamento dos códigos etc. O apresentador também mostra que é possível fazer a limpeza e manutenção de arquivos que não serão mais úteis, e por fim, mostra o Grafana, ferramenta que

possibilita programar mensagens de emergência enviadas via email em caso de alguma métrica obtida pelas observações das outras ferramentas.

E por último, foram apresentadas algumas técnicas para lidar com segurança no Airflow.

A primeira foi a de criptografia para dados sensíveis. Dados como senhas são salvos em textos claros do airflow, além disso o banco de dados por padrão não possui nenhuma restrição, permitindo que qualquer um possa realizar uma consulta a fim de pegar uma senha. Por isso, é recomendado ativar o secure mode no arquivo de configuração do airflow para impossibilitar a exploração do banco de dados, além de ativar a criptografia com Fernet para dados sensíveis. Além disso, o instrutor recomenda que seja feita uma rotação das chaves, ou seja, trocar as chaves de criptografia de tempos em tempos a fim de dificultar o acesso de intrusos. Ele também ensina a desabilitar a visualização das variáveis através da interface do airflow mexendo no arquivo de configuração do airflow, mais especificamente ativando o campo de hide sensitive variable fields. E por fim, o instrutor ensina a criar um sistema de autenticação com authenticate = True, e então criar um usuário e dividir os usuários por funções, cada um com acesso a partes diferentes do Airflow, sendo essa na minha opinião uma das partes mais importantes de todo o curso, pois é essencial dividir as tarefas principalmente quando o grupo de Devs é grande, não só por questões de segurança mas também por facilidade e praticidade.

3. Conclusão

Em resumo, o Airflow é uma ferramenta muito útil para organizar grande volumes de tarefas, tendo muito mais a oferecer do que simplesmente agendamento de tarefas, contendo atualmente muitas ferramentas para auxiliar sua implementação através de interfaces gráficas, implementação de dashboards com relatórios das tarefas e diferentes arquiteturas e executores para atender desde projetos menores até projeto de grandes corporações, sendo extremamente versátil e escalável.

4. Referências

Videoaulas do curso do Airflow do Card.